

DA Platform Architecture

How the Development Agent portal reaches the DA Services backend, how DA Services depends on the DA Registry, and which external systems it integrates with. Compiled from the FSD (D3), the Part 1 HLD, the team's architecture diagram (3 Oct) and the staging UI.

OAN Ethiopia · Development Agent Registry & Services · draft for team review · 6 Oct 2026

- ☐ DA Services (Frappe) — Nikky & Pushkar
- ☐ DA Registry (OpenG2P Gen 2) — Yash
- ☐ Gateway and authentication
- ☐ External systems (owned by other teams)

1. System context

The UI never talks to a database or to another system directly. Every request goes through Kong to DA Services. DA Services owns the operational data (visits, tasks, leave, KPIs) and fetches the DA's identity and posting from the DA Registry. Farmer, grievance and survey data stay in the systems that own them; DA Services keeps only references.

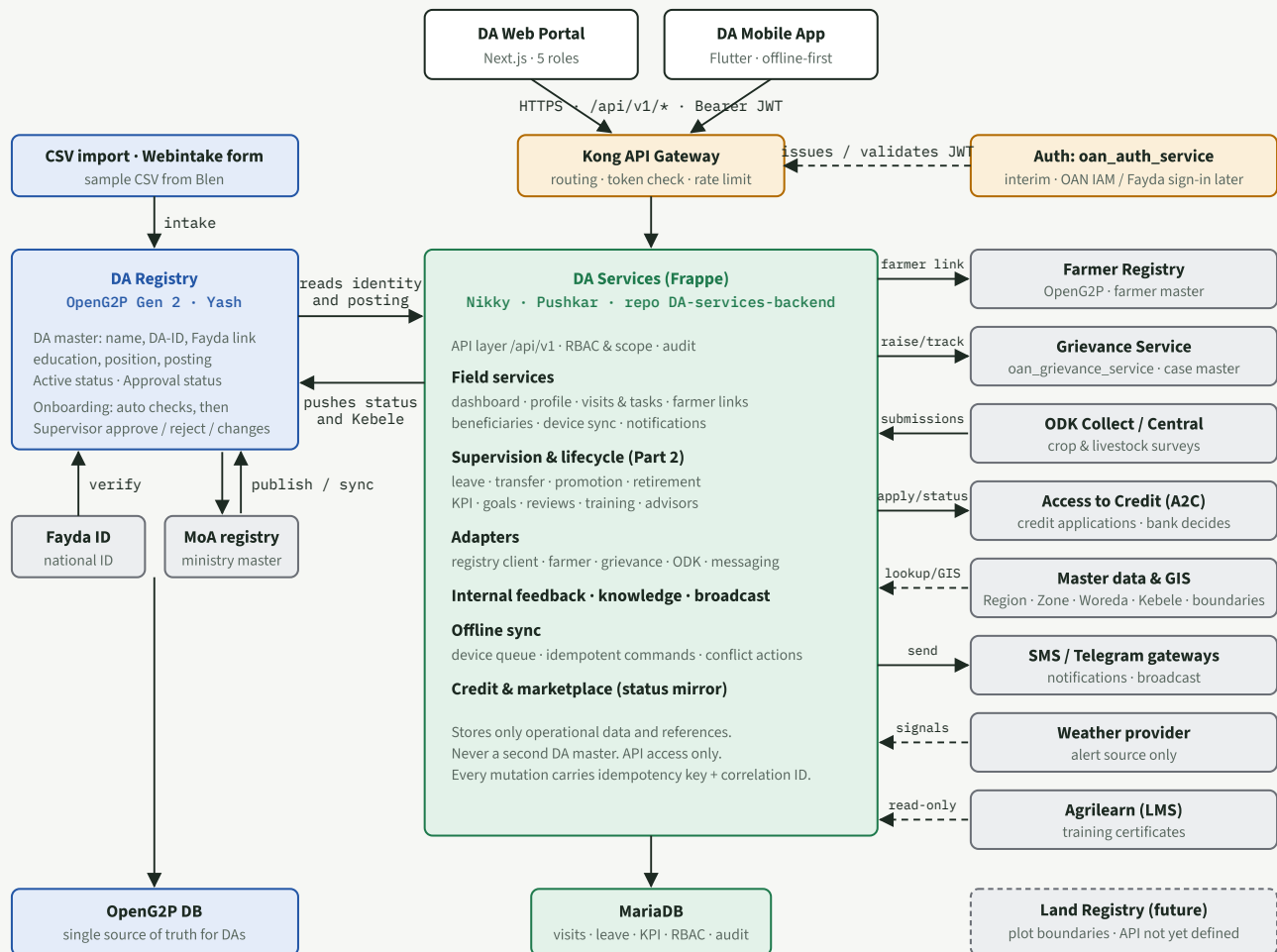


Figure 1. The portal and mobile app reach only DA Services (through Kong). DA Services reads DA identity and posting from the Registry and pushes approved status or Kebele changes back; all other systems are reached through adapters and hold their own master data. Solid arrows are request/response; dashed arrows are periodic sync or read-only. Fayda and MoA are reached through the Registry, not by DA Services. Land Registry is planned but its API is still under discussion.

2. Who owns which data

The rule from the HLD: a fact lives in exactly one authoritative system; everyone else keeps a reference. DA Services therefore stores what it creates and fetches what it only displays.

DATA	AUTHORITATIVE OWNER	WHAT DA SERVICES KEEPS
DA identity: name, DA-ID, Fayda link, gender, birth date (EC), contacts	DA Registry	Only <code>da_id</code> as a reference; fetched per request (short cache allowed)

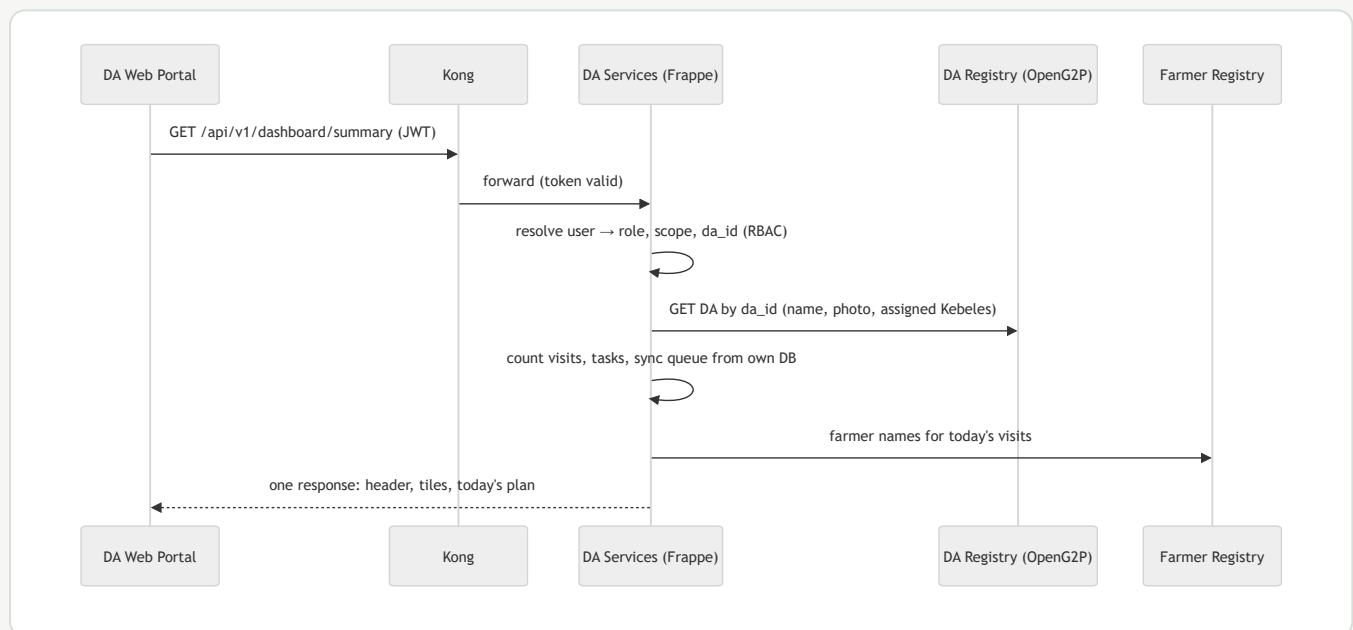
DATA	AUTHORITATIVE OWNER	WHAT DA SERVICES KEEPS
DA posting: Region, Zone, Woreda, Kebele assignment	DA Registry	Codes on the RBAC assignment for scope checks
Active status (Active / On-leave / Separated)	DA Registry	Sets it through lifecycle outcomes (leave, retirement, reactivation)
Approval status, Fayda status, onboarding workflow	DA Registry	Reads it; shows Supervisor queue as pass-through
Users, roles, Region/Woreda scope, DA-ID binding	DA Services	DA RBAC Assignment (built)
Visits, tasks, outcomes, GPS, attachments	DA Services	Own tables
Leave requests and balances, lifecycle transitions	DA Services	Own tables; approved outcome pushed to Registry
KPIs, goals, reviews, training records, certificates, incentives	DA Services	Own tables
Internal feedback issues, notifications, device sync queue, audit	DA Services	Own tables
Farmer profile, household, plots	Farmer Registry	farmer_id + DA-farmer link; beneficiary classification history
Grievance case, status, responses	Grievance Service	external_case_id + status mirror + offline queue
Crop and livestock survey submissions	ODK Central	Visit ↔ submission correlation and validation result
Region / Zone / Woreda / Kebele lists, Kebele boundaries	Master data / GIS	Read-only lookup table, refreshed by sync; geofence result stored on the assignment
Credit applications, marketplace payments	Access to Credit / payment providers	Application reference + status mirror; the bank decides, DA Services only displays
Training and certification records	Agrilearn	Read-only mirror with verification status; uploaded external certificates are DA Services' own

DATA	AUTHORITATIVE OWNER	WHAT DA SERVICES KEEPS
Weather and emergency signals	Weather provider	Inbound signal in the alert inbox; the broadcast built from it is DA Services' own
Plot boundaries	Land Registry (future)	Boundary captured in the app stays "Awaiting sync" until the API exists
Login, tokens, password reset	oan_auth_service	Maps the token's user to role, scope and DA-ID

3. Three requests, end to end

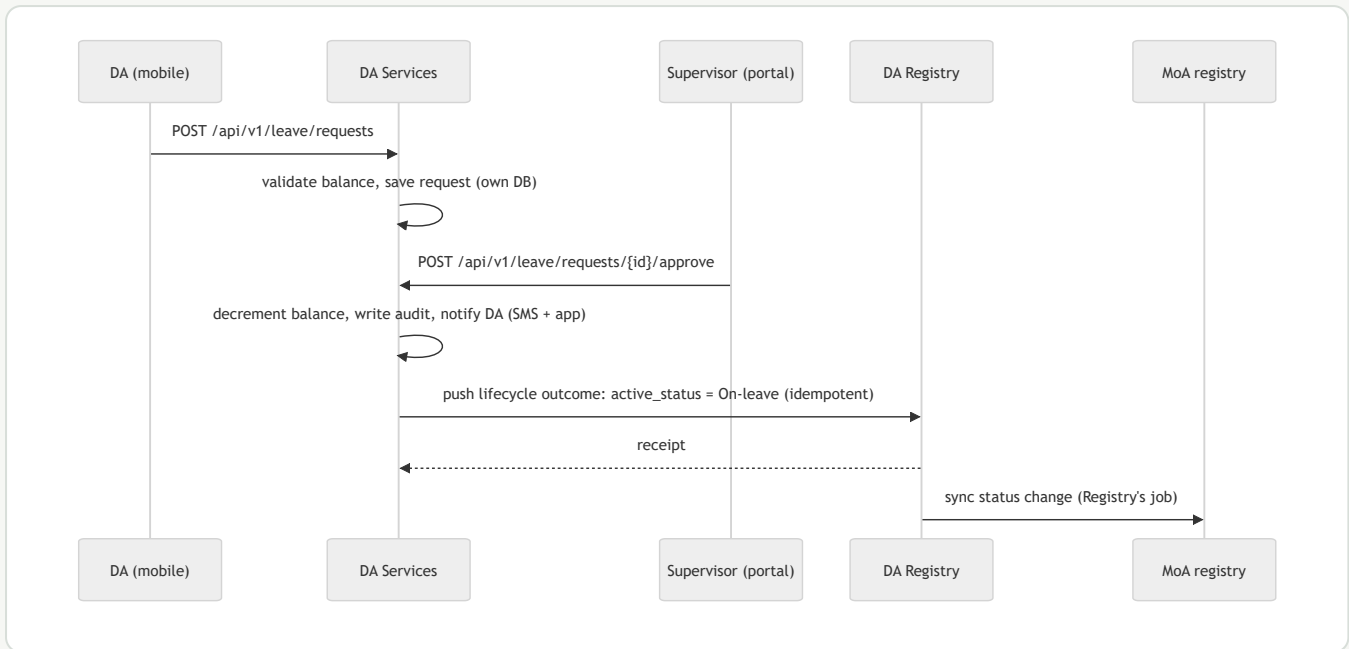
Where the data comes from is easiest to see on real pages.

3.1 DA opens the dashboard



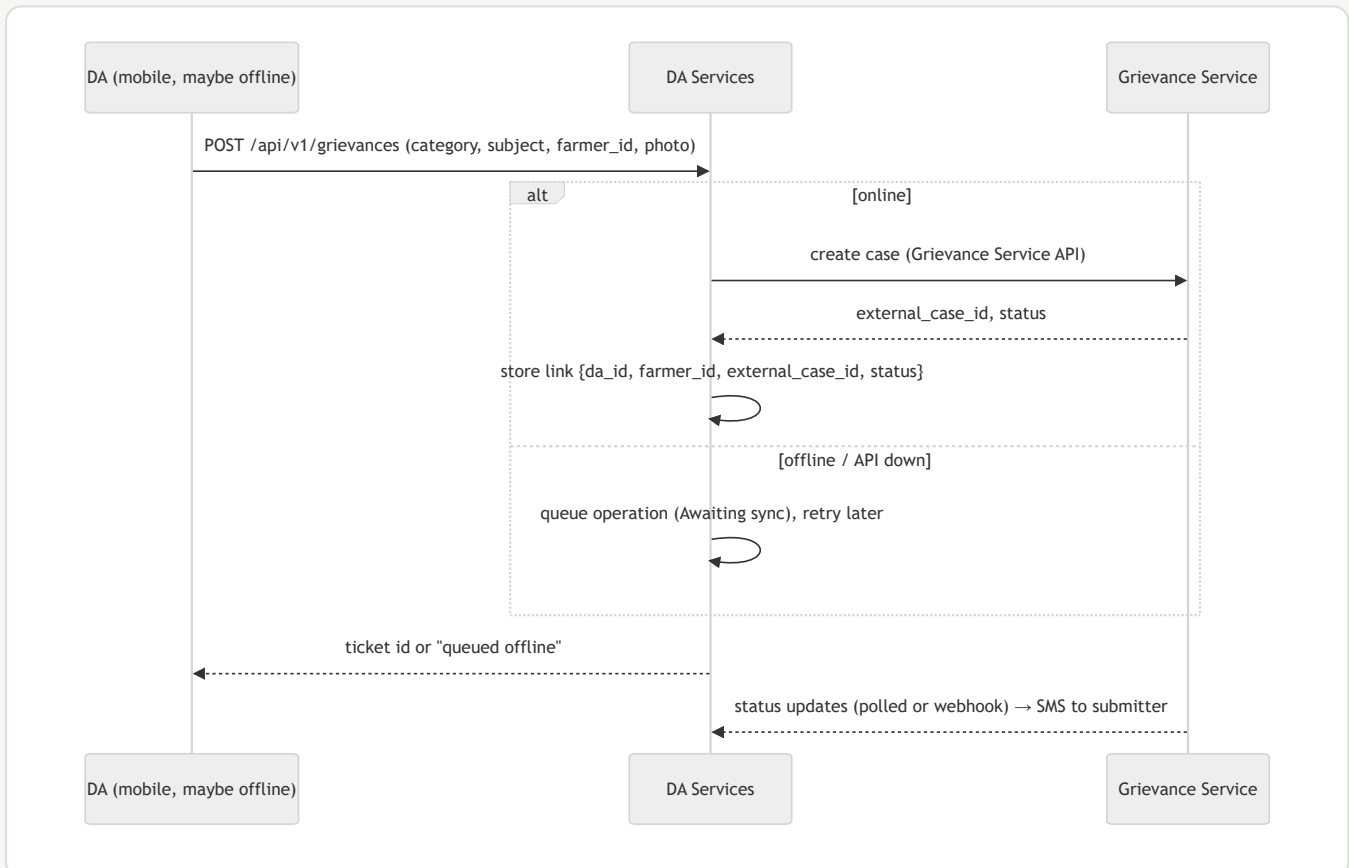
About 90% of the dashboard is DA Services' own data. The Registry contributes only the DA's name and posting.

3.2 Supervisor approves a leave request



Leave is DA Services' record. The Registry only learns the resulting status, because the Active Status column on the master registry is defined by the FSD as "set by Part 2 lifecycle".

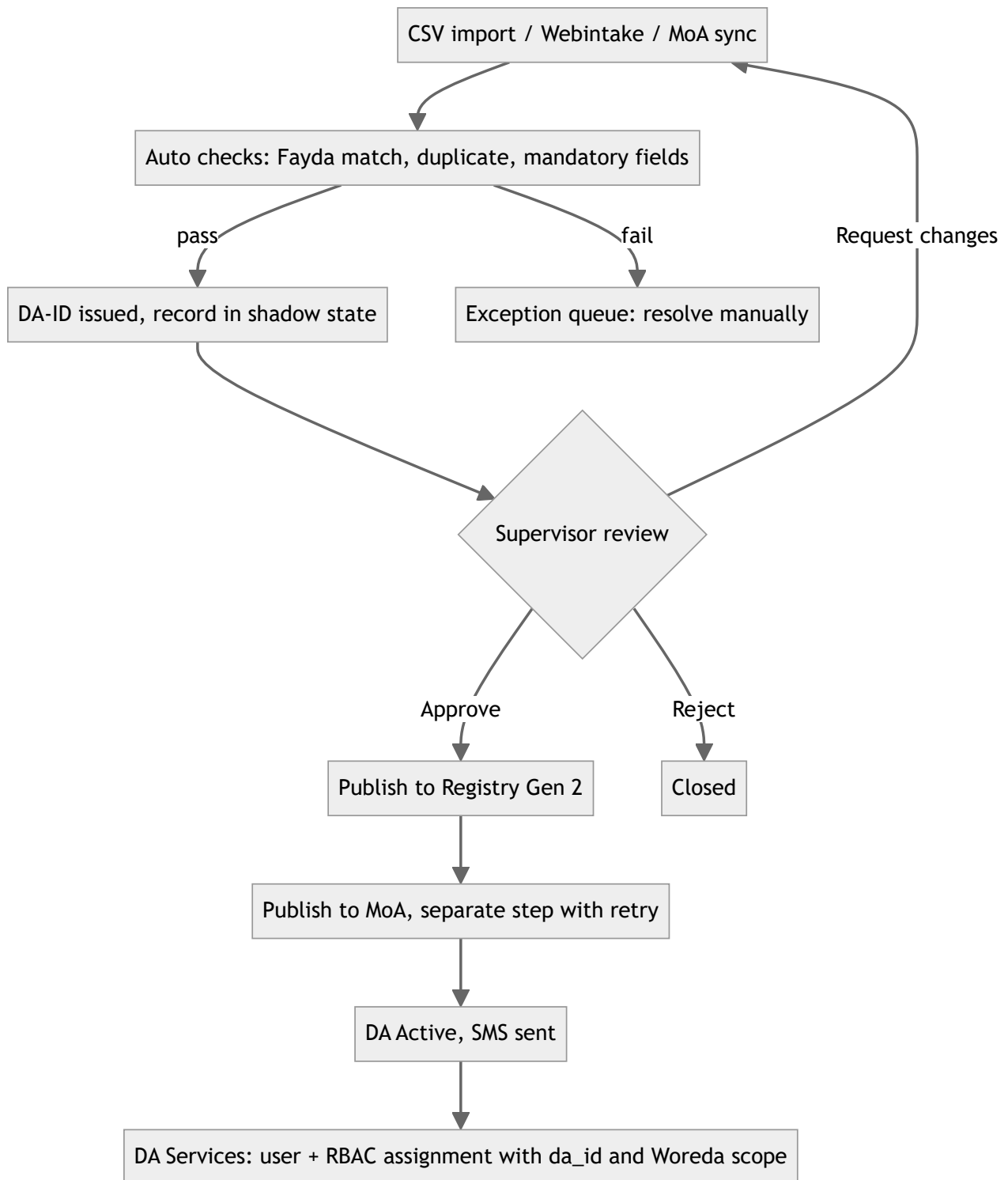
3.3 DA raises a grievance for a farmer



Triage, response, escalation and closure happen in the Grievance Service. DA Services shows them read-only and keeps only the link and the offline queue.

4. How a DA gets into the system

Onboarding is the Registry's workflow (Yash). DA Services only picks up the result: an approved, active DA becomes a user with a role, a scope and a DA-ID.



Machines do the checks; a person makes the decision. Approval and publication are separate states, and MoA publication has its own retry and reconciliation.

5. Roles and scope

Authorisation is deny-by-default and enforced in the backend on every call. A role says which actions exist; the RBAC assignment says over which Region, Woreda or DA.

ROLE	SCOPE	CAN	CANNOT
Development Agent	Own DA-ID	Own profile, farmers, visits, leave requests, feedback, grievances	Approve anything, see other DAs
Supervisor	Own Woreda	Approve onboarding and lifecycle, assign Kebeles, approve leave, team views	Act outside own Woreda, manage users
Executive	Own Region	Regional dashboards, SLA roll-ups, reports	Any edit, approval or assignment
OAN Administrator	All	Users, roles, scopes, settings, sync config, audit	Delete immutable audit records
Communications Officer	Broadcast domain	Broadcasts, alerts, knowledge articles	See or edit DA identity, approvals

6. Team ownership

Yash — DA Registry (OpenG2P)

- DA entity and validation rules (Shikha's field list)
- Webintake form, CSV import, MoA sync
- Fayda verification, duplicate check, DA-ID issuance
- Supervisor review workflow and publication
- Registry table, search, filter, exceptions
- Registry APIs consumed by DA Services

Nikky & Pushkar — DA Services (Frappe)

- All APIs behind the portal and mobile app (~150 endpoints estimated)
- Auth integration, RBAC and scope (DA-27)
- Dashboard, profile, visits, tasks, farmers, sync, notifications
- Leave, lifecycle, KPI, reviews, training, advisors, surveys
- Adapters: Registry, Farmer Registry, Grievance, ODK, messaging

7. Open decisions

- **Registry integration path.** Does DA Services call OpenG2P directly or a Registry API layer? Determines the registry client, status push-back and whether events or polling are used.
- **Login.** `oan_auth_service` as interim (same as Grievance) vs central OAN IAM. Does Kong validate the token or only route?

- **Fayda sign-in.** Needs relying-party credentials, a sandbox and HTTPS on staging. Planned as a second sign-in once available.
- **Registry pages in the portal** (`/agents` , `/approvals` , `/assignments` , `/registry-sync`). Served through DA Services as pass-through, or by OpenG2P's own UI?
- **Field list.** Fayda ID, mandatory fields, dropdown values, Zone level, joining date; naming (Name / Father / Grandfather vs First / Last) and birth date (EC month-year vs full Gregorian) differ between Shikha's list and the UI.
- **Supervisors per Woreda** and whether any one of them may approve, or a primary approver exists.
- **Master data source** for Region / Zone / Woreda / Kebele.

8. Where things stand

Environment

Frappe v16 bench in Docker, site `da-services.localhost`, `SETUP.md` for the team

Code

5 roles, DA RBAC Assignment, scope checks, permission hooks, mock Registry client, first endpoint, 25 tests

Delivery

PR #1 to develop open for review; Jira DA-27 subtasks created (4 done, 9 to do)

Next

Wire `oan_auth_service` + `/me`, RBAC admin API, then DA-role APIs (dashboard, profile, visits)

Sources: `EOAN_SPEC_FRS_DA_Registry_D3` (study guide), `EOAN_SPEC_Design_DA_Registry_Part1_D1`, DA Services Part 2 brief, team architecture diagram, DA Registry Fields (Shikha), staging portal. Numbers of users per role are estimates, not from the documents.